



QA Tester

Kontakt

TELEFON

515 836 183

E-MAIL

kaplukasz@gmail.com

LOKALIZACJA

Radzyń Podlaski

PORTFOLIO

lukasztm.github.io/Portfolio

Atuty

- Dokładna dokumentacja testowa
- Analityczne myślenie
- Czytelne raportowanie defektów
- Powtarzalny workflow QA
- Otwartość na naukę automatyzacji i AI

Stack QA

Manual QA

Test cases

Bug reports

Jira / Xray

TestLink

Postman

SQL

Python

unittest

PyTest

Selenium

Claude Code

Obsługa AI

Narzędzia

Jira, Xray, TestLink, Bugzilla, Postman, JMeter, Oracle SQL Developer, Chrome DevTools, PyCharm, Git/GitHub, ShareX, Claude Code.

Języki

Angielski — A2

Zainteresowania

analiza błędów

nowe narzędzia QA

gry strategiczne

zagadki logiczne

Łukasz Kap

Testuję manualnie, automatyzuję w Pythonie i Selenium, a teraz rozwijam tę pracę o narzędzia AI — od obsługi klienta po Claude Code.

● Manual QA

● Automatyzacja testów

● AI & Claude Code

Profil zawodowy

Kandydat na stanowisko Junior QA Tester, rozwijający się w kierunku automatyzacji testów oraz pracy wspieranej narzędziami AI. Portfolio pokazuje pełny proces pracy testera: analizę wymagań, tworzenie przypadków testowych, wykonanie testów, raportowanie defektów oraz retesty. Łączę podstawy Manual QA z SQL, API/Postman, Pythonem, unittest i Selenium WebDriver, a dodatkowo rozwijam umiejętności obsługi klienta wspieranej AI oraz podstawy pracy z asystentem Claude Code.

Projekty QA i portfolio

Manual QA

Pełny cykl testów manualnych: analiza wymagań, przypadki testowe, wykonanie testów i retesty.

SQL / API

Zapytania SQL, testowanie API w Postmanie oraz analiza żądań w Chrome DevTools.

Automatyzacja

Skrypty w Pythonie, testy jednostkowe unittest oraz automatyzacja UI w Selenium WebDriver.

AI w pracy testera

Obsługa klienta z wykorzystaniem AI oraz podstawy pracy z Claude Code przy zadaniach testerskich.

Kompetencje QA

Manual testing

- Analiza wymagań: user stories, kryteria akceptacji i ryzyka.
- Scenariusze testowe: przypadki, dane testowe i rezultaty oczekiwane.
- Testy: funkcjonalne, eksploracyjne, regresyjne, walidacyjne oraz UI/UX.
- Bug reporty: kroki, actual/expected result, priorytet, załączniki i retest.

Automatyzacja i wsparcie

- Python: funkcje, warunki, listy, słowniki, wyjątki i walidatory danych.
- unittest: TestCase, asercje, scenariusze pozytywne i negatywne.
- Selenium WebDriver: lokatory, formularze, upload pliku, alerty, dropdowny.
- SQL i API: SELECT, JOIN, GROUP BY, Postman, REST, JSON, statusy HTTP.

Obsługa klienta AI i Claude Code

● Obsługa klienta z AI

- Sprawna, empatyczna komunikacja z klientem zgłaszającym problem.
- Wsparcie pierwszej linii z użyciem chatbotów i asystentów AI.
- Szybkie formułowanie jasnych, rzeczowych odpowiedzi na zgłoszenia.

● Claude Code (podstawy)

- Uruchamianie i podstawowa konfiguracja w terminalu.
- Zlecanie prostych zadań: analiza pliku, uruchomienie testów, raport.
- Wsparcie przy debugowaniu i podstawowym code review.

Kursy QA

2026 · ALX

Kurs Tester Automatyzujący

Python dla testerów, HTML/CSS, lokatory, Selenium WebDriver, PyTest, Page Object, raportowanie oraz podstawy Selenium Grid.

2025 · ALX

Kurs Tester Oprogramowania z elementami AI

Pisanie przypadków testowych, raportowanie błędów, API/Postman, SQL, SDLC, Agile/Scrum, testy eksploracyjne.

Edukacja

2011 — Liceum Ogólnokształcące



Zeskanuj i zobacz portfolio

Python dla testera

- Typy danych, zmienne, operatory, warunki if/elif/else i pętle.
- Listy, krotki, zbiory i słowniki jako dane testowe.
- Funkcje z parametrami, *args i **kwargs — proste helpery.
- Wyjątki: ValueError, TypeError, KeyError, FileNotFoundError.
- Praca z plikami: odczyt, zapis, logi i raporty testowe.

Testy jednostkowe i dane

- unittest: TestCase, asercje, scenariusze +/-.
- Walidatory danych: telefon, PESEL, cena, login, hasło.
- Regex jako wsparcie walidacji pól tekstowych.
- Organizacja projektu testowego w PyCharm.
- Debugowanie kodu krok po kroku.

Selenium WebDriver

- Lokatory: ID, CSS Selector, XPath, tag, name, link text.
- Automatyzacja formularzy: input, checkbox, radio, dropdown.
- Obsługa alertów JavaScript i komunikatów na stronie.
- Upload pliku przez input type=file z send_keys().
- Raport z testu: data, status PASSED/FAILED/ERROR.

PyTest + Page Object

- Konfiguracja: requirements.txt, pytest.ini, conftest.py.
- Struktura folderów: pages oraz tests.
- Page Object dla strony logowania: pola, przycisk, komunikat.
- Testy logowania, checkboxów i listy rozwijanej.
- Uruchamianie testów z terminala, pytest-repeat.

HTML, CSS i analiza strony

- Struktura HTML: nagłówki, akapity, linki, obrazy, formularze.
- Atrybuty: id, class, href, src, alt, name, value.
- Czytanie kodu strony pod kątem testowania.
- Podstawy CSS oraz atrybut style przy analizie wyglądu.

Selenium Grid i dalszy rozwój

- Testy na różnych przeglądarkach i środowiskach.
- Podstawy Grid: hub, node, zdalne uruchomienie testu.
- Kolejny krok: CI/CD, Allure, GitHub Actions, Jenkins.
- Dalszy rozwój: API testing, SQL w testach, Page Object.

Porozmawiajmy o współpracy

Chętnie opowiem więcej o moim portfolio QA i pokażę, jak wygląda mój warsztat testera — manualny, automatyzacja i praca z AI.

TELEFON

515 836 183

E-MAIL

kaplukasz@gmail.com

PORTFOLIO

lukasztm.github.io/Portfolio